

❖ Function

1. What is a Function?

- A function is a reusable block of code that performs a specific task. Instead of writing the same code multiple times, we write it once inside a function and call it whenever needed.

- Why Do We Need Functions?

- Without Functions
 - Code repetition
 - Difficult to maintain
 - Difficult to debug
 - Large code size
- With Functions
 - Code Reusability
 - Better Readability
 - Easy Maintenance
 - Easy Testing
 - Modular Programming

2. Syntax of Function

```
def function_name(parameters):  
    # function body  
    return value
```

3. Types of Functions

1) Built-in Functions

- print()
- len()
- sum()
- max()
- min()
- type()
- input()

2) User-defined Functions

```
def greet():  
    print("Good Morning")  
  
greet()
```

4. Types of User-defined Functions

1) No Arguments, No Return Value

```
def greet():  
    print("Good Morning")  
  
greet()
```

2) Arguments, No Return Value

```
def greet(name):  
    print("Hello", name)  
  
greet("user")
```

3) No Arguments, Return Value

```
def addition():  
    return 10 + 20  
  
result = addition()  
  
print(result)
```

4) Arguments and Return Value

```
def addition(a, b):  
    return a + b  
  
print(addition(10, 20))
```

➤ Parameters and Arguments

Parameters	Arguments
Variables in function definition	Actual values passed while calling
Placeholder	Real Data

5) Return Statement

```
def square(n):  
    return n*n  
  
print(square(5))
```

➤ Difference Between print() and return

print()	return
Displays output	Sends value back
Cannot reuse result	Result can be stored
Used for displaying	Used for processing

5. Types of Return

1) Function without Return

- Returns None by default.

```
def greet():  
    print("Good Morning")  
  
result = greet()  
  
print(result)
```

2) Function with Single Return Value

```
def square(n):  
    return n * n  
  
result = square(5)  
  
print(result)
```

3) Function with Multiple Return Values

- Python returns them as a **tuple**.

```
def calculate(a, b):  
    return a+b, a-b, a*b  
  
add, sub, mul = calculate(20, 10)  
  
print(add)  
print(sub)  
print(mul)
```

6. Function Arguments

- Arguments are the actual values passed to a function when it is called.

Types of Arguments

1) Positional Arguments

- Arguments are assigned according to their position. The order must be the same as the parameters.

```
def student(name, age):  
    print(name)  
    print(age)  
  
student("demo", 22)
```

2) Keyword Arguments

- Arguments are passed using the parameter name. Order does not matter.

```
def student(name, age):  
    print(name)  
    print(age)  
  
student(age=22, name="demo")
```

3) Default Arguments

- A parameter is assigned a default value. If no argument is passed, the default value is used.

```
def greet(name="Guest"):
    print("Welcome",name)

greet()

greet("demo")
```

4) Variable-Length Arguments (*args)

- *args allows a function to accept any number of positional arguments. Inside the function, they are stored as a tuple.

```
def total(*numbers):
    for i in numbers:
        print(i)

total(10,20,30,40)
```

5) Variable-Length Keyword Arguments (**kwargs)

- **kwargs allows a function to accept any number of keyword arguments. Inside the function, they are stored as a dictionary.

```
def student(**details):
    print(details)

student(name="demo",age=22)

student(name="Rahul",age=20,city="Pune")
```

7. Nested Functions

- A Nested Function is a function defined inside another function. The inner function can only be accessed within the outer function.

```
def outer():  
    def inner():  
        print("Inner Function")  
    inner()  
outer()
```